HealthNexus Feature Guide

Every feature, the files that build it, and how it works. Written so any team member can explain the project.

Scope: all frontend and backend features **except Auth and Onboarding**, which have their own guide. Also covers the ML anomaly module and what is not built yet.

Last updated: 8 October 2026

How to read this guide

- Section 1 is the big picture: the two kinds of users, how a click travels from the screen to the database, and the folder map.
- Section 2 is one block per feature. Each has: what the user sees, a table of files (frontend and backend), the steps that happen, and the rules worth knowing.
- Section 3 covers the ML module. Section 4 lists what is not built yet. Section 5 is a one-line cheat sheet and a glossary.
- File paths are written like `backend/app/services/scoring.py`. Backend paths start at `backend/app/` and frontend paths at `frontend/src/`.

1. The big picture

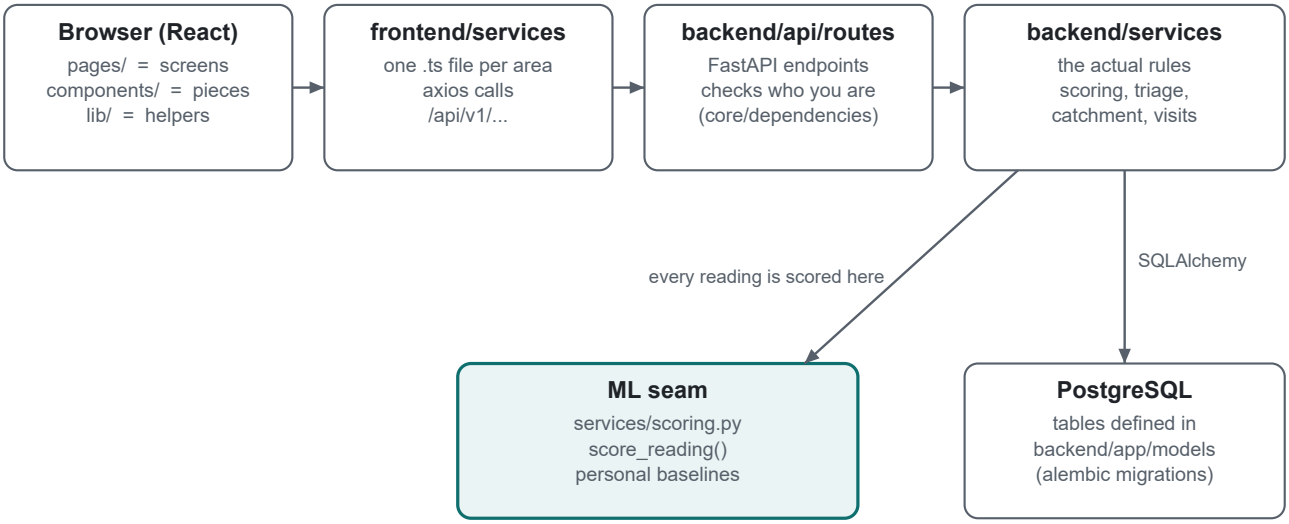
HealthNexus connects **citizens** who log their health readings with **PHC workers** (staff at a Primary Health Centre) who review anything unusual and visit people. The app checks each reading, flags the unusual ones, alerts the citizen, and puts the flag in the health worker's to-do list.

The two roles

Role	Code name	What they do	Pages start with
Citizen	`user`	Logs vitals, sees their dashboard, history, alerts and visits.	`/app`
PHC worker	`phc_worker`	Reviews flagged citizens in their area, books and records visits, sees reports and the risk map.	`/phc`
Admin	`admin`	Approves PHC workers. The admin console is still a "Coming soon" page.	`/admin`

How a click reaches the database

Every feature follows the same path. Once you can explain this once, you can explain every feature.



1. A **page** in `frontend/src/pages/` shows the screen. It loads data with the `useResource` hook (`lib/useResource.ts`), which handles loading, errors and retry.
2. The page calls a function in `frontend/src/services/` (for example `getSummary()` in `health.ts`). That file is the only place that knows the URL. It uses the shared axios client in `services/api.ts`, which adds the login token and refreshes it when it runs out.
3. The request reaches a **route** in `backend/app/api/routes/`. All routes are joined in `api/router.py` under `/api/v1`. A dependency such as `require_onboarding_user` or `require_catchment_worker` (`core/dependencies.py`, `services/catchment.py`) checks who is calling before any code runs.
4. The route uses **services** in `backend/app/services/` for the real rules (scoring, triage, visits), and reads or writes **tables** through the models in `backend/app/models/`.
5. The answer is shaped by a **schema** in `backend/app/schemas/` (Pydantic), so the frontend always gets the same fields.

Folder map

Folder	What lives there
`backend/app/api/routes/`	Endpoints, one file per area: vitals, alerts, phc, schedule, settings, profile (plus auth and onboarding).
`backend/app/api/views.py`	Small helpers that turn database rows into responses, shared by several routes (alerts, visits).
`backend/app/services/`	Business rules. Most important: readings, scoring, triage, catchment, localities, visits, visit_state, consent.
`backend/app/models/`	One file per database table (SQLAlchemy). Enums such as visit status are in models/enums.py.
`backend/app/schemas/`	Request and response shapes. health.py for citizen data, phc.py for worker pages.
`backend/alembic/versions/`	Database migrations 0001 to 0009, in the order the tables were added.
`backend/app/seeds/`	Scripts that fill a demo database: facilities, admin, a PHC worker, a citizen, and a 27-person demo catchment.
`backend/tests/`	pytest tests for rules, patterns, modules, mailer (and auth, onboarding).
`frontend/src/pages/`	One file per screen. citizen/ and phc/ hold the role-specific pages.
`frontend/src/components/app/`	Pieces used inside the app: AppShell (menu and badges), TrendChart, AlertList, ui.tsx (cards, buttons, badges), phc.tsx (worker pieces).
`frontend/src/components/landing/`	The public landing page sections (Hero, FeatureShowcase, Footer and others).
`frontend/src/services/`	API calls: health.ts (citizen), phc.ts (worker), account.ts (settings and profile), api.ts (shared client).
`frontend/src/lib/`	Helpers: routes.ts (all URLs, and where each role lands), severity.ts, format.ts, useResource.ts, useForm.ts.
`ml/`	The anomaly detection research code and notebooks (Section 3).

Words used everywhere

Word	Meaning
Reading	One set of vitals a citizen logged at one moment. Stored in `health_records`.
Flag	A reading the scorer called unusual. The score is stored in `processed_data` with `is_anomaly = true`.
Open flag	A flag no worker has reviewed or dismissed yet (its resolution is empty).
Severity	critical (score of 0.75 or more), high (0.5 or more), moderate (0.25 or more), low. Defined once in `services/triage.py`.
Catchment	All citizens linked to the same health centre as the worker (same `phc_id`). A worker sees only their catchment.
Citizen ref	A short ID such as `HN-1A2B3C4D`, made from the user id, so it is the same on every page.
Locality	The area name a citizen gave (`users.area`). Used for reports and the map.

2. Features

Citizen features come first, then PHC worker features, then the shared ones (settings, profile, app shell).

Citizen features

2.1. Log vitals (the most important feature)

A citizen enters whatever they measured: blood pressure, heart rate, temperature, oxygen, breathing rate, glucose or weight, plus optional symptoms. Straight away they see whether the reading was flagged and why.

Side	File	What it does
Frontend	`pages/citizen/SubmitVitals.tsx`	The form. Fields are grouped (blood pressure, vitals, other). Checks typed values before sending, and shows the result screen with "How this reading was checked".
Frontend	`components/app/ModelInsights.tsx`	ScoreBreakdown: per vital, what it was compared with (normal range, the person's usual, or both) and the result.
Frontend	`services/health.ts`	getMetrics() loads the list of vitals and their ranges; submitVitals() sends the reading.
Backend	`api/routes/vitals.py`	GET /health/metrics returns the vitals list. POST /health/vitals saves and scores a reading.
Backend	`services/metrics.py`	THE list of vitals: name, unit, normal range and plausible range. Also the common symptoms.
Backend	`services/readings.py`	record_reading(): saves the reading, scores it, and creates an alert if it is flagged.
Backend	`services/scoring.py`	score_reading(): the anomaly model. Published ranges first, then each person's own baseline (see the ML Report).
Backend	`services/triage.py`	Turns the score into a severity and a plain sentence such as "Blood pressure 167/103 mmHg, well above the normal range".
Database	`health_records`, `processed_data`, `alerts`	Tables this feature reads or writes.

What happens, step by step

1. The page calls `GET /health/metrics` so the form shows the same vitals and ranges as the backend. Nothing is duplicated in the frontend.
2. The citizen submits. The backend first checks the citizen has health tracking turned on (a consent). If it is off, the request is refused.
3. Each value is checked against its **plausible** range. For example, a heart rate of 900 is a typo, not an anomaly, so the user is asked to fix it.
4. `record_reading()` saves the reading in `health_records`, then loads up to 120 earlier readings and calls `score_reading()`.
5. The score (0 to 1), the flag, and how much each vital contributed are saved in `processed_data`.
6. If flagged, an alert is written for the citizen. It says whether a health worker will see it, or that nobody is assigned yet.
7. The response tells the page: flagged or not, severity, which vital caused it, what each vital was compared with, and any "watch" vitals (unusual for the person but not yet confirmed).

Rules worth knowing (useful when someone asks)

- Normal range vs plausible range: outside normal means the reading can be flagged; outside plausible means it is rejected as a typo.
- A personal baseline needs at least 7 earlier readings (MIN_BASELINE_READINGS). Before that, the app says so and compares with published ranges.
- From 7 readings of a vital, the person's own usual starts to count; from 21, it is used alone. Safety limits (for example fever 38.0 C or SpO2 92%) always flag.
- A value inside the normal range that is only unusual for this person needs a second unusual reading in a row before it flags.
- Every flag can explain itself: the model stores how each vital was judged (processed_data.details), plus the basis and model version.

2.2. Citizen dashboard

The citizen's home page: a greeting, their latest value for each vital compared with their usual value, a 30-day trend chart, open alerts, their next visit, and their health centre.

Side	File	What it does
Frontend	`pages/citizen/Dashboard.tsx`	Builds the page, including the "How well we know your normal" card. Also groups repeated alert sentences so five similar alerts do not read as a wall of text.
Frontend	`components/app/ModelInsights.tsx`	BaselineProgress: a bar per vital from learning to personal.
Frontend	`components/app/TrendChart.tsx`	Recharts line chart for one vital. The shaded band is the normal range; flagged readings are bigger dots and say "flagged" in the tooltip.
Frontend	`services/health.ts`	getSummary() and getMetrics().
Backend	`api/routes/vitals.py`	GET /health/summary collects everything the dashboard needs in one request. GET /health/baseline gives the model's progress per vital.
Backend	`services/readings.py`	baseline_of(): the model's own personal baseline (median and robust spread), only from 7 values.
Backend	`services/catchment.py`	workers_serving(): how many workers cover the citizen's health centre.
Database	`health_records, processed_data, alerts, scheduled_visits, valid_phc_facilities`	Tables this feature reads or writes.

What happens, step by step

1. The page asks for the summary and the vitals list at the same time.
2. The backend loads readings from the last 90 days (up to 500). For each vital it takes the newest value and compares it with the earlier values: below, normal or above, plus the personal average if there are enough readings.
3. It adds the last 30 days of readings for the chart, counts (total readings, last 7 days, open flags, unread alerts), the 5 newest unread alerts, the next booked visit and the health centre.
4. The page shows a card per vital and the chart for the selected vital.

Rules worth knowing (useful when someone asks)

- One request for the whole page keeps it fast and consistent.
- If there are not enough readings yet, the page shows how many more are needed (readings_until_baseline).

2.3. History

A list of every reading, newest first, with filters (one vital, a time window, flagged only) and the option to delete a reading.

Side	File	What it does
Frontend	`pages/citizen/History.tsx`	Filters (7, 30, 90 or 365 days, or all), paging of 20 rows, delete button.
Frontend	`services/health.ts`	getHistory() and deleteReading().
Backend	`api/routes/vitals.py`	GET /health/history (filters and paging). DELETE /health/readings/{id}.
Database	`health_records`, `processed_data`, `audit_logs`	Tables this feature reads or writes.

What happens, step by step

1. The page sends the chosen filters. The backend joins each reading with its score, so each row knows if it was flagged and its severity.
2. Each flagged row also gets its plain-language reason and whether a worker has reviewed it.
3. Delete: the backend only finds readings that belong to the caller, removes the score and the reading, and writes an `audit_logs` entry.

Rules worth knowing (useful when someone asks)

- Someone else's reading id simply returns "not found", so ids cannot be probed.
- If a visit was booked from a deleted reading, the visit stays; it just loses its link to the reading.

2.4. Citizen alerts and visits

One page with the citizen's visits (needs attention, upcoming, past) and their alerts, followed by community advisories.

Side	File	What it does
Frontend	`pages/citizen/Alerts.tsx`	Sorts visits into sections by their state, for example "Needs your attention" for visits that passed with no outcome.
Frontend	`components/app/AlertList.tsx`	Shows personal alerts, then community advisories. Marks shown alerts as read, which clears the badge.
Frontend	`services/health.ts`	getAlerts(), getMySchedule(), markRead().
Backend	`api/routes/alerts.py`	GET /alerts (own alerts plus advisories from the last 30 days), GET /alerts/unread-count, POST /alerts/mark-read.
Backend	`api/routes/schedule.py`	GET /schedule/mine: the citizen's visits.
Database	`alerts`, `scheduled_visits`	Tables this feature reads or writes.

What happens, step by step

1. The page loads the schedule and the alerts together.
2. Visits are placed in sections using their state (see 2.9). Missed visits older than 30 days are hidden.
3. Unread alerts are marked read on the server once shown, so the menu badge goes back to zero.

Rules worth knowing (useful when someone asks)

- Community advisories have no owner, so they are never counted in the unread badge.
- mark-read only updates the caller's own alerts, so ids of other people's alerts change nothing.

PHC worker features

The catchment rule, which every worker page depends on. A worker sees only citizens whose `phc_id` is the same as theirs. This is written once in `services/catchment.py` and every worker query uses it. If a citizen is outside the catchment, the answer is exactly the same as for a citizen who does not exist (404). A worker who is not linked to any health centre gets a clear "No health centre linked" page (409) instead of an empty dashboard, because an empty page would look like a quiet day.

2.5. PHC dashboard

The worker's home page: the facility name, headline numbers, 8-week trends, and the people who most need review today.

Side	File	What it does
Frontend	`pages/phc/PhcDashboard.tsx`	Number cards, small weekly charts (KpiCard), and the 6 worst citizens to review, with a severity filter.
Frontend	`components/app/phc.tsx`	CitizenWorkCard (one card per citizen), NoFacility, and workerState (shared loading, error and 409 handling).
Frontend	`services/phc.ts`	getDashboard(), getOverview(), getWorklist().
Backend	`api/routes/phc.py`	GET /phc/dashboard (counts), GET /phc/overview (4 weekly trends over 8 weeks), GET /phc/worklist.
Backend	`services/catchment.py`	Limits every count to the worker's catchment.
Database	`users, health_records, processed_data, scheduled_visits`	Tables this feature reads or writes.

What happens, step by step

1. The page loads three things at once: the dashboard counts, the weekly overview and the worklist.
2. Dashboard counts: citizens, citizens flagged, open flags by severity, readings and active citizens in the last 7 days, engagement (needs attention, never logged, silent, ok), visits today and visits needing an outcome.
3. Overview: readings, citizens who submitted, flags raised and flags cleared, per calendar week (weeks start on Monday, Indian time). The current week is marked as partial.

Rules worth knowing (useful when someone asks)

- "Silent" means no reading for 14 days (SILENT_AFTER).
- Severity counting in SQL is built from the same bands as triage.py, so the numbers cannot disagree.

2.6. Alerts worklist and review (the worker's main job)

Every open flag grouped by person: one citizen is one piece of work, worst first. The worker can mark a person reviewed, dismiss with a reason, or book a visit. A second view shows all flags from the last 14 days.

Side	File	What it does
Frontend	`pages/phc/PhcAlerts.tsx`	Search by name or HN reference (waits until typing pauses), filter by locality, switch between the worklist and recent flags.
Frontend	`components/app/phc.tsx`	CitizenWorkCard lists that person's open flags; ResolveControls gives "Reviewed" in one click, while "Dismiss" asks for a reason.
Frontend	`services/phc.ts`	getWorklist(), resolveCitizen(), markSeen(), getFeed(), bookVisit().

Side	File	What it does
Backend	`api/routes/phc.py`	GET /phc/worklist, POST /phc/worklist/citizens/{id}/resolve (clear all of a person's flags), POST /phc/queue/{id}/seen (clear one flag), GET /phc/queue, GET /phc/alerts (feed).
Backend	`services/triage.py`	Severity and the plain-language reason for each flag.
Database	`processed_data, health_records, users, scheduled_visits`	Tables this feature reads or writes.

What happens, step by step

1. The worklist groups open flags by citizen and sorts by the worst score. Each card shows up to 10 flags, the symptoms, and any visit already booked by this worker.
2. Reviewed or dismissed: the backend saves who reviewed it, when, the decision, the reason and a note. The flag stops being open.
3. Clearing one flag that is already cleared returns a conflict (409), so two workers cannot both clear it.

Rules worth knowing (useful when someone asks)

- A dismissal must have a reason, because a dismissal without a reason cannot be audited.
- Grouping by person means the same citizen never appears as five separate rows to act on.
- The menu badge shows how many citizens have open flags (GET /phc-alerts/unread-count).

2.7. Citizens directory

Everyone registered at the worker's health centre, with search, locality and status filters.

Side	File	What it does
Frontend	`pages/phc/Citizens.tsx`	The table, filters and paging (50 at a time).
Frontend	`services/phc.ts`	getDirectory().
Backend	`api/routes/phc.py`	GET /phc/citizens. The _directory() helper joins each citizen with their reading count, last reading and open flags.
Backend	`services/localities.py`	locality_of(): an empty area becomes "Unspecified".
Database	`users, user_profiles, health_records, processed_data`	Tables this feature reads or writes.

What happens, step by step

1. Each citizen gets one status: **attention** (has open flags), **never_logged**, **silent** (nothing for 14 days) or **ok**.
2. The list is sorted in that order, so people who need help come first.
3. The response also lists the localities and how many citizens each has, which fills the locality filter.

2.8. Citizen record

One citizen's full picture for the worker: open flags, latest values against their own baseline, health profile, a chart per vital, every submission, visits and alerts, for a chosen window (30, 90, 180 or 365 days).

Side	File	What it does
Frontend	`pages/phc/CitizenRecord.tsx`	The record page. Checks the id in the URL is a valid UUID before asking the server. Flagged readings show "vs their baseline" and an expandable "Why it was flagged".
Frontend	`services/phc.ts`	getRecord(userId, days).
Backend	`api/routes/phc.py`	GET /phc/citizens/{id}/record?days=90.

Side	File	What it does
Backend	`services/catchment.py`	get_catchment_citizen(): 404 if the person is not in this worker's catchment.
Database	`users, user_profiles, health_records, processed_data, alerts, scheduled_visits`	Tables this feature reads or writes.

What happens, step by step

1. The backend loads the window's readings (up to 2,000) with their scores, the alerts sent, visits in the window and upcoming visits.
2. Visits from before the window that still have no outcome are reported next to it, not mixed into it.
3. The timeline is cut to the newest 200 entries, and the page says so when that happens.

2.9. Visits and schedule

Workers book a home visit for a citizen (often straight from a flag), then mark it done, nobody there, cancelled, or move it. Both sides get notified.

Side	File	What it does
Frontend	`pages/phc/Schedule.tsx`	The worker's visits in groups (due now, today, upcoming, needs outcome), with buttons for each action and recent outcomes.
Frontend	`components/app/phc.tsx`	Booking from a work card; suggests the next slot at the start of the worker's working hours tomorrow.
Frontend	`services/phc.ts`	bookVisit(), getSchedule(), updateVisit().
Backend	`api/routes/phc.py`	POST /phc/visits (book), GET /phc/visits (upcoming).
Backend	`api/routes/schedule.py`	GET /schedule (worker view), GET /schedule/mine (citizen view), PATCH /schedule/{id} (complete, miss, cancel, move).
Backend	`services/visit_state.py`	The one definition of a visit's state.
Backend	`services/visits.py`	The notification messages for booked, cancelled, moved and missed visits.
Database	`scheduled_visits, alerts`	Tables this feature reads or writes.

What happens, step by step

1. Booking checks the citizen is in the catchment and the time is allowed (within 90 days ahead). If the citizen already has a visit booked, booking is refused: move that one instead.
2. Both the citizen and the worker get an alert, worded for each of them.
3. The worker records an outcome. Done is not allowed for a later day; "nobody there" only after the time has passed.
4. If 4 hours pass with no outcome, the visit becomes overdue, and both sides get a one-time reminder.

Rules worth knowing (useful when someone asks)

- Stored status is only what someone decided: scheduled, completed, cancelled, missed.
- State = status + the clock: upcoming, today, due_now (passed, within 4 hours), overdue. A visit is in exactly one state, so the counts never overlap.
- Days follow Indian time (Asia/Kolkata), not UTC.
- The missed reminder is claimed with one database UPDATE, so two page loads at once cannot send it twice.

2.10. Reports

Totals only, with no names: readings per day and a table by locality, for 7, 30, 90 or 180 days.

Side	File	What it does
Frontend	`pages/phc/Reports.tsx`	Daily readings chart and the locality table.
Frontend	`services/phc.ts`	getReports(days).
Backend	`api/routes/phc.py`	GET /phc/reports?days=30. Counts whole local days: "last 30 days" is today and the 29 days before it.
Backend	`services/localities.py`	suppress(): merges small localities so no one can be identified.
Database	`users, health_records, processed_data`	Tables this feature reads or writes.

What happens, step by step

1. The backend counts readings, flags and active citizens per day and per locality.
2. Any locality with fewer than 3 citizens (MIN_CELL) is merged into "Other localities".

Rules worth knowing (useful when someone asks)

- Why merge small localities: "1 of 1 citizens in Luz is flagged" would name that person as surely as a list would.

2.11. Risk map

An OpenStreetMap map with a circle per locality, coloured by the share of its citizens with an open flag, plus a ranking.

Side	File	What it does
Frontend	`pages/phc/RiskMap.tsx`	Leaflet map (react-leaflet). Circle size grows with the number of citizens; the colour always has a text label as well.
Frontend	`services/phc.ts`	getMap().
Backend	`api/routes/phc.py`	GET /phc/map and GET /phc/locality-risk.
Backend	`services/localities.py`	locality_risk(): counts flagged citizens per locality, averages their location, and applies the same small-locality merge.
Database	`users, processed_data`	Tables this feature reads or writes.

What happens, step by step

1. For each locality: citizens, citizens with an open flag, share = flagged / citizens.
2. Level: **high** at 30% or more, **elevated** at 15% or more, **some** above 0, otherwise **none**.
3. Merged and "Unspecified" groups have no map position, so they appear only in the ranking.

2.12. Worker notifications

The worker's own messages: visits they booked, changes, and visits waiting for an outcome.

Side	File	What it does
Frontend	`pages/Notifications.tsx`	The list, reusing AlertList.
Frontend	`services/health.ts`	getAlerts().
Backend	`api/routes/alerts.py`	GET /alerts (the same endpoint citizens use; it returns the caller's own alerts).

Side	File	What it does
Backend	`services/visits.py`	Writes these messages.
Database	`alerts`	Tables this feature reads or writes.

What happens, step by step

1. The page loads the worker's alerts and marks them read, which clears the Notifications badge.

Shared features (citizens and workers)

2.13. Settings

A tabbed page. Citizens see Account, Preferences, Privacy, Support and Health centre. Workers also see a Work tab for their working hours.

Side	File	What it does
Frontend	`pages/Settings.tsx`	All tabs: AccountTab, PreferencesTab, PrivacyTab, SupportTab, CentreTab, WorkTab.
Frontend	`services/account.ts`	One function per settings endpoint.
Backend	`api/routes/settings.py`	Preferences, consents and consent history, about, export, password, deactivate, concerns, health centre.
Backend	`services/consent.py`	Consent history is append-only: a change adds a new row, and the newest row is the current answer.
Backend	`services/catchment.py`	facility_serves_district(): a citizen can only choose an active centre in their own district.
Database	`users, consent_records, concerns, audit_logs, scheduled_visits, alerts`	Tables this feature reads or writes.

What happens, step by step

1. **Account:** change password, see signed-in devices, deactivate (needs the password; signs out everywhere and cancels booked visits).
2. **Preferences:** theme and notification options, saved as a partial update (only the changed parts are sent).
3. **Privacy:** turn each consent on or off (health tracking, PHC worker access, community surveillance, research, marketing), see the history, and download all your data as one JSON file (passwords and tokens are never included).
4. **Support:** raise a concern (data misuse, incorrect flag, harassment, access request, other) and track it.
5. **Health centre:** switch to another centre in your district. Visits booked by the old centre are cancelled and those workers are told, because they can no longer see you.
6. **Work (workers only):** working hours, used to suggest visit times.

Rules worth knowing (useful when someone asks)

- Turning off health tracking blocks new readings (see 2.1).
- Changes such as switching health centre or deleting a reading are written to audit_logs with the IP address.

2.14. Profile

Name, phone, bio, location, date of birth (and department for workers), a profile picture and cover image, and changing email address.

Side	File	What it does
Frontend	`pages/Profile.tsx`	The form, image pickers (PNG, JPEG, WebP) and the email change box.
Frontend	`pages/ConfirmEmail.tsx`	Opened from the email link to confirm the new address.
Frontend	`services/account.ts`	getProfile(), updateProfile(), uploadImage(), removeImage(), requestEmailChange(), confirmEmailChange().
Backend	`api/routes/profile.py`	GET and PATCH /profile, POST and DELETE /profile/avatar and /profile/cover, POST /profile/email and /profile/email/confirm.
Backend	`main.py`	Serves uploaded images at /api/v1/media.
Backend	`services/mailer.py`	Sends the confirmation email.
Database	`users, user_profiles`	Tables this feature reads or writes.

What happens, step by step

1. Images are checked by their actual file bytes (not just the name) and saved under a random file name, so a URL cannot be guessed.
2. Uploading a new image removes the old one.
3. Email change needs the current password, and only takes effect after the link in the email is clicked.

2.15. App shell, navigation and badges

The frame around every signed-in page: the menu for each role, unread badges and the theme.

Side	File	What it does
Frontend	`components/app/AppShell.tsx`	Builds the menu for citizens or workers, applies the saved theme, and checks the badges every 60 seconds.
Frontend	`components/ProtectedRoute.tsx`	Only lets the right role into each page.
Frontend	`lib/routes.ts`	Every URL, and destinationFor(): where each kind of user lands after signing in.
Frontend	`main.tsx`	The route table: which page belongs to which URL and role.
Frontend	`components/app/ui.tsx`	Shared look: PageHeader, Card, Btn, SeverityBadge, VisitBadge, Loading, ErrorNote, Empty, Stat, Tabs.
Backend	`api/routes/alerts.py`	GET /alerts/unread-count (citizen badge), GET /phc-alerts/unread-count (worker: unread messages and citizens flagged).

What happens, step by step

1. After signing in, AppShell loads preferences (theme) and starts checking the badge counts.
2. Citizens see: Home, Log vitals, History, Alerts. Workers see: Dashboard, Alerts (badge = citizens flagged), Citizens, Schedule, Reports, Risk map, Notifications (badge = unread).

2.16. Landing page

The public home page for visitors who are not signed in.

Side	File	What it does
Frontend	`pages/LandingPage.tsx`	Puts the sections together.
Frontend	`components/landing/*`	Hero, FeatureShowcase, CommunityInsights, CommunityShowcase, PhoneMockup, PrivacyBanner, FinalCta, Footer and others.
Frontend	`content/landing.ts`	The text on the page, kept separate from the layout.

What happens, step by step

1. Signed-in users are sent straight to their own home page instead (RedirectIfAuthenticated).

Demo data (seeds)

To demo the app, the scripts in `backend/app/seeds/` fill the database: health centres, an admin, a PHC worker, one citizen, and `seed_demo_catchment.py`, which creates 27 citizens with 14 days of daily readings, some flagged, in the worker's catchment. Run it with `python -m app.seeds.seed_demo_catchment` after the worker seed. It uses `record_reading()`, so demo data is scored exactly like real data, and it can be run again safely.

3. ML anomaly detection module

The anomaly model is **live in the app** (`services/scoring.py`, version personal-baseline-v1). The full story, with metrics, challenges and the reasons behind each decision, is in the separate **HealthNexus ML Report**.

File	What it does
`ml/healthnexus_ml/preprocessing.py`	Cleans the LifeSnaps Fitbit dataset (71 people): removes days the watch was not worn, blanks impossible values, and counts each person's history for each signal.
`ml/healthnexus_ml/anomaly.py`	GlobalModel (Isolation Forest on everyone), personal baselines (rolling median and spread of the person's own last 60 days), the blend between them, and score_days(), which returns a score, a flag, an alert and a reason for each day.
`ml/healthnexus_ml/train.py`	Trains the global model on all users and saves models/global_anomaly.joblib.
`ml/notebooks/01_preprocessing.ipynb`	Why each cleaning step was done: 7,410 raw days become 4,766 clean days.
`ml/notebooks/02_anomaly_model.ipynb`	Builds and tests the model. On fake illness-like changes the personal baseline reaches an AUC of 0.83 to 0.90; the global model alone is near guessing (0.57 to 0.61).
`ml/notebooks/03_alert_rule.ipynb`	Alert only after 2 flagged days in a row: false alarms drop from about 1.1 to 0.4 per user per month.
`ml/notebooks/04_app_scorer.ipynb`	Evaluates the app's own scorer on real data: false flags 24.6% to 1.5%, moderate episodes caught 16% to 68%.
`ml/tests/`	11 pytest tests on cleaning and scoring.
`ml/healthnexus_ml/clustering.py, features.py`	Empty. Outbreak clustering is not started.

How it plugs in

The app has one replaceable spot: `score_reading(user_id, vitals, history)` in `services/scoring.py`. It returns a score from 0 to 1, flagged or not, how much each vital contributed, the basis, and per-vital details. Triage, alerts and every worker page use that shape. The research model was trained on wearable signals the app does not collect, so the **method** was ported (published ranges plus personal baselines), not the trained Isolation Forest.

4. Not built yet

Good to know so nobody claims these work in a demo.

What	Where	State
AI health assistant (chat)	`api/routes/assistant.py`, `services/assistant/`, `features/assistant/`	Empty files. The chat_sessions and chat_messages tables exist (migration 0006) but nothing uses them.
Outbreak clustering	`api/routes/clusters.py`, `ml/healthnexus_ml/clustering.py`	Empty. The community_data table exists but is unused.
Admin console	`/admin`	Shows a "Coming soon" page.
Community advisories authoring	`alerts (alert_type = community)`	Advisories are shown if they exist, but no screen creates them yet.

5. Cheat sheet: explain each feature in one line

Feature	One-line explanation
Log vitals	The citizen enters readings; the backend rejects typos, scores the reading, and alerts them and their health centre if it is unusual.
Dashboard	One request returns the latest value of each vital compared with the person's own usual value, a 30-day chart, alerts and the next visit.
History	Every reading with filters; deleting one is limited to your own readings and is written to the audit log.
Citizen alerts	The person's alerts and visits, sorted by what needs attention; viewing them clears the badge.
Catchment	A worker only ever sees citizens linked to their own health centre; everyone else is "not found".
PHC dashboard	Headline counts, 8-week trends and the worst cases to review today, all limited to the catchment.
Worklist	Open flags grouped by person, worst first; review in one click, dismiss only with a reason.
Citizens directory	Everyone at the centre, labelled needs attention, never logged, silent for 14 days, or ok.
Citizen record	One person's flags, baseline, charts, readings, visits and alerts for a chosen window.
Visits	Book from a flag, record the outcome; the state comes from the status plus the clock, and both sides are notified.
Reports and risk map	Totals by day and locality with no names; localities under 3 people are merged so no one can be identified.
Settings	Password, preferences, consents with full history, data export, concerns, and switching health centre.
Profile	Personal details, images checked by their file bytes, and email change confirmed by a link.
Anomaly model	Starts from published ranges and learns each person's normal; safety limits always flag, and an in-range change needs two readings in a row.

Questions people often ask

- **Why not flag against normal ranges forever?** People differ. A resting heart rate of 85 can be normal for one person and a warning sign for another. After 7 readings the app can compare a person with themselves.
- **How do you stop workers seeing the wrong people?** Every worker query goes through services/catchment.py, and "not yours" looks exactly like "does not exist".
- **How is privacy handled in reports?** Totals only, small localities merged (MIN_CELL = 3), consent history kept forever, and data export excludes secrets.
- **Why is a dismissal reason required?** So every cleared flag can be checked later: who cleared it, when and why.
- **Why is "overdue" not stored in the database?** It depends on the clock. Working it out when the page loads means it can never be out of date.